Name:NJeevan Sai
Register Number: 192525169

CO5 – AT1
MLA0201 – Fundamentals of Machine Learning

Annexure A

Constraint-Based Problem Solving

Question 1 (20 Marks): Reinforcement Learning for Warehouse Robot Navigation Scenario

An e-commerce company uses an autonomous warehouse robot to transport packages between storage racks and dispatch areas. The robot must reach the destination while avoiding obstacles, minimizing travel time, and conserving battery power.

Constraints

- The robot can move only in four directions (Up, Down, Left, Right).
- Obstacles cannot be crossed.
- Each movement consumes one unit of battery.
- The robot must reach the destination within 25 moves.
- Battery capacity is limited to 30 units.
- Some paths provide positive rewards, while blocked paths incur penalties.
- The environment is stochastic, and some actions may not produce the intended movement.

Task

Design a Reinforcement Learning solution by selecting an appropriate exploration strategy and applying either **Value Iteration** or **Policy Iteration** to determine the optimal navigation policy. Justify your approach based on the given constraints.

Question 2 (20 Marks): Multi-Armed Bandit for Online Advertisement Selection Scenario

A digital marketing company displays one of several online advertisements to website visitors. The objective is to maximize the click-through rate while learning customer preferences over time.

Constraints

- Five advertisements are available.
- Only one advertisement can be displayed per visitor.
- The click probability of each advertisement is initially unknown.
- The system must balance exploration and exploitation.
- The total advertising budget allows only 10,000 user interactions.
- The solution should minimize cumulative regret while maximizing total rewards.

Task

Formulate the problem as a **K-Armed Bandit** problem. Select an appropriate exploration strategy (e.g., ε-greedy, UCB, or Thompson Sampling) and justify how it satisfies the given constraints. Explain how the reward mechanism influences advertisement selection.

Question 3 (20 Marks): Sampling Strategy for Machine Learning Model Development Scenario

A healthcare analytics company is developing a disease prediction model using a dataset containing one million patient records. Limited computational resources prevent training on the complete dataset.

Constraints

- Training memory is limited to 8 GB RAM.
- The sample must accurately represent all patient categories.
- The prediction accuracy should remain above 95%.
- Model confidence must be at least 95%.
- The solution should minimize sampling bias and computational cost.
- The selected learning approach should satisfy sample complexity and generalization requirements.

Task

Recommend an appropriate sampling method (e.g., Simple Random Sampling, Stratified Sampling, or Monte Carlo Sampling) and justify your choice. Analyze how **sample complexity**, **VC dimension**, **Occam's Learning Principle**, and **accuracy-confidence boosting** influence the model's performance under the given constraints.

1. Reinforcement Learning for Warehouse Robot Navigation

1. Problem Understanding

The warehouse robot needs to move from a **starting position to a destination** while:

- Avoiding obstacles.
- Reaching the destination within **25 moves**.
- Conserving battery, with a maximum capacity of **30 units**.
- Maximizing positive rewards.
- Avoiding paths with penalties.
- Handling a **stochastic environment**, where an action may not always produce the intended movement.

This can be modeled as a **Markov Decision Process (MDP)**.

MDP Components

- **State (S):** Robot's current position, remaining battery, and number of moves.
- **Actions (A):** Up, Down, Left, Right.
- **Reward (R):**
  - Positive reward for reaching the destination.
  - Small negative reward for every movement to encourage shorter paths.
  - Large negative reward for hitting/attempting blocked paths.
- **Transition Probability P:** Represents the probability that an action results in the intended movement.
- **Discount factor (γ):** A value such as 0.9 can be used to prefer immediate rewards.

2. Suitable Exploration Strategy

I would use **ε-greedy exploration**. In ε-greedy:

- With probability **ε**, the robot chooses a random action for exploration.
- With probability **1 − ε**, it chooses the action with the highest estimated value.

For example:

ε = 0.1

- 10% → explore a different action.
- 90% → exploit the best known action.

The value of ε can be gradually reduced during learning:

$$\epsilon_t = \epsilon_0 \times decay^t$$

This allows the robot to explore initially and become more confident about the best path later.

Why ε-greedy?

It is suitable because the robot needs to discover safe and efficient routes while avoiding unnecessary exploration. A **decaying ε** provides more exploration initially and more exploitation later.

3. Value Iteration

I would use **Value Iteration** because the warehouse environment has stochastic transitions.

The Bellman optimality equation is:

$$V(s) = \max_a [R(s,a) + \gamma \sum_{s'} P(s', | sa) V(s')]$$

Where:

- $V(s)$= value of the current state.
- $R(s,a)$= reward for taking action a.
- $P(s', | sa)$= probability of reaching next state s'.
- γ = discount factor.

Example

Suppose the robot is at state **S1** and considers moving Right.

If:

- 0.8 probability → moves Right.
- 0.2 probability → moves Down because of stochastic movement.

Then:

$$Q(S, 1Right) = R(S, 1Right) + \gamma[0.8V(S_{Right}) + 0.2V(S_{Down})]$$

The robot selects the action having the highest value.

4. Handling the Constraints

Constraint	**RL Solution**
Four directions	Action space = Up, Down, Left, Right
Obstacles	Blocked states/actions receive large negative rewards
Battery limit 30	Battery included in the state
Maximum 25 moves	Move counter included in the state
Positive rewards	Reward for reaching destination
Penalties	Penalty for blocked/inefficient paths
Stochastic actions	Transition probabilities model uncertainty

The state can therefore be represented as:

$$S = (p, o, sitionbatterymoves)$$

A terminal state is reached when:

- Robot reaches destination, or
- Moves = 25, or
- Battery becomes 0.

5. Optimal Policy

After Value Iteration converges, the optimal policy is:

$$\pi^*(s) = \arg\max_a [R(s,a) + \gamma \sum_{s'} P\ (s', \mid sa)V(s')]$$

Thus, for every position, the robot knows the best direction to take.

The resulting policy should:

- Reach the destination.
- Avoid obstacles.
- Minimize unnecessary movement.
- Conserve battery.
- Handle stochastic movement.

Conclusion

Value Iteration with decaying ε-greedy exploration is appropriate for this problem because it handles uncertainty, rewards, penalties, obstacles, and limited resources. Including battery and move count in the state ensures that the robot learns a policy that satisfies the 30-unit battery and 25-move constraints.

2. Multi-Armed Bandit for Online Advertisement Selection

1. Problem Formulation

This problem can be modeled as a **5-Armed Bandit problem**.

There are **5 advertisements**, and each visitor receives exactly one advertisement.

Let:

$$A = \{A, d_1, A, d_2, Ad_3Ad_4Ad_5\}$$

Each advertisement has an unknown click probability:

$$p_1, p_2, p_3, p_4, p_5$$

The system must learn these probabilities while maximizing the total number of clicks.

The total number of interactions is:

$$T = 10{,}000$$

2. Reward Mechanism

The reward can be defined as:

$$R = \begin{cases} 1 & \text{if user clicks the advertisement} \\ 0 & \text{if user does not click} \end{cases}$$

Therefore, the estimated click-through rate of advertisement $i$is:

$$\hat{p_i} = \frac{\text{number of clicks}}{\text{number of times Ad}_i\text{was displayed}}$$

Example

Suppose:

Advertisement	Displays	Clicks	Estimated CTR
Ad 1	2,000	180	9%
Ad 2	2,000	150	7.5%
Ad 3	2,000	220	11%
Ad 4	2,000	120	6%
Ad 5	2,000	160	8%

Ad 3 currently has the highest estimated CTR.

3. Selected Strategy: UCB

I would choose the **Upper Confidence Bound (UCB)** strategy.

The UCB score is:

$$UCB_i = \hat{p}_i + \sqrt{\frac{2 \ln t}{n_i}}$$

Where:

- $\hat{p}_i$= estimated CTR of advertisement i.
- n_i= number of times advertisement $i$has been displayed.
- t= total number of interactions.

The advertisement with the highest UCB value is selected.

4. Exploration and Exploitation

UCB naturally balances exploration and exploitation.

Exploitation

An advertisement with a high estimated CTR has a high:

$$\hat{p}_i$$

Therefore, it is likely to be selected.

Exploration

An advertisement that has been displayed fewer times has a larger:

$$\sqrt{\frac{2 \ln t}{n_i}}$$

bonus.

Therefore, UCB gives less-tested advertisements an opportunity to be selected.

This is useful because an advertisement that currently has a low estimated CTR may actually have a high true CTR but may not have received enough trials.

5. Cumulative Regret

Regret is the difference between the reward from always selecting the best advertisement and the reward obtained by the algorithm.

$$Regret(T) = Tp^* - \sum_{t=1}^{T} R_t$$

where $p^*$is the true click probability of the best advertisement.

The goal is to minimize:

$$Regret(10,000)$$

UCB is appropriate because it reduces unnecessary exploration while still testing uncertain advertisements.

6. Why UCB is Suitable

Constraint	**UCB Solution**
Five advertisements	Five arms
Unknown click probability	Estimates CTR from rewards
One ad per visitor	Selects exactly one arm
Exploration required	Confidence bonus encourages exploration
Exploitation required	High CTR ads receive high scores
10,000 interactions	UCB works efficiently over finite trials
Minimize regret	Gradually concentrates on the best advertisement

Conclusion

The problem is a **5-armed bandit with 10,000 trials**. I recommend **UCB** because it does not require a manually selected ε value and automatically balances exploration and exploitation using the confidence term. The reward of **1 for a click and 0 for no click** directly influences advertisement selection. Over time, the system should display high-performing advertisements more frequently while continuing to test uncertain advertisements.

3. Sampling Strategy for Machine Learning Model Development

1. Problem Understanding

The healthcare company has:

$$N = 1{,}000{,}000$$

patient records.

Training on the complete dataset is computationally expensive and exceeds the available resources. Therefore, a representative sample must be selected.

The important requirements are:

- Maximum **8 GB RAM**.
- All patient categories must be represented.
- Accuracy should remain above **95%**.
- Model confidence should be at least **95%**.
- Sampling bias should be minimized.
- Computational cost should be reduced.
- The sample should provide good generalization.

2. Recommended Sampling Method: Stratified Sampling

I recommend **Stratified Random Sampling**.

In this method, the complete dataset is divided into meaningful groups called **strata**, and samples are selected from each group.

For example, strata may represent:

- Disease categories.
- Age groups.
- Gender categories.
- Other important patient classes.

If the original dataset contains:

- 60% Category A
- 25% Category B
- 15% Category C

A representative sample should approximately maintain the same proportions.

For a sample of 100,000 records:

- Category A → 60,000
- Category B → 25,000
- Category C → 15,000

This prevents important categories from being accidentally underrepresented.

3. Why Not Simple Random Sampling?

Simple Random Sampling gives every record an equal chance of selection.

However, it can accidentally produce an unbalanced sample.

For example, if a rare disease represents only 2% of the population, random sampling may select too few patients from that category.

This can reduce the model's ability to predict rare diseases.

Therefore, **Stratified Sampling is safer for healthcare data**.

4. Sample Complexity

Sample complexity refers to the amount of training data required for a model to learn a reliable relationship between inputs and outputs.

A simplified generalization relationship can be written as:

$$m = O\left(\frac{d + \ln(1/\delta)}{\epsilon^2}\right)$$

Where:

- m= required sample size.
- d= model complexity.
- ϵ= acceptable error.
- δ= probability of failure. For **95% confidence**:

$$1 - \delta = 0.95$$

Therefore:

$$\delta = 0.05$$

A sufficiently large and representative sample reduces the probability of poor generalization.

5. VC Dimension

VC dimension (Vapnik–Chervonenkis dimension) measures the complexity or learning capacity of a model.

A model with a very high VC dimension can represent more complex patterns but generally requires more training data.

Therefore:

- High VC dimension → more samples required.
- Lower appropriate VC dimension → fewer samples required.

For this problem, the company should choose a model whose complexity is appropriate for the available sample size.

A very complex model trained on a small sample may **overfit**, resulting in high training accuracy but poor performance on unseen patients.

6. Occam's Learning Principle

Occam's Learning Principle states that when multiple models explain the data well, the simpler model is generally preferred.

For this problem:

- Avoid unnecessarily complex models.
- Use a sufficiently simple model that captures important disease patterns.
- This reduces overfitting.
- It also reduces computational requirements.

Thus, Occam's principle supports using a model with suitable complexity rather than an unnecessarily complicated model.

7. Accuracy and Confidence Boosting

The target is:

$$Accuracy \geq 95\%$$

and

$$Confidence \geq 95\%$$

To improve accuracy and confidence:

1. Use stratified sampling.

2. Maintain representation of all patient categories.
3. Remove duplicate and poor-quality records.
4. Perform proper preprocessing.
5. Use cross-validation.
6. Tune model hyperparameters.
7. Evaluate on an independent test set.

For example, if the model achieves:

$$Accuracy = 96.2\%$$

and confidence is:

$$Confidence = 95\%$$

then the stated performance requirements are satisfied.

However, **confidence should be calibrated and evaluated**, not simply assumed from accuracy.

8. Memory and Computational Cost

Instead of processing all:

$$1{,}000{,}000$$

records, a representative sample can be selected, such as:

$$100{,}000 - 200{,}000$$

records, depending on feature size and model complexity.

The exact sample size should be determined using validation and sample-complexity analysis rather than choosing an arbitrary percentage.

A smaller representative dataset:

- Requires less RAM.
- Reduces training time.
- Reduces computational cost.
- Can still provide strong generalization if properly sampled.

9. Complete Recommended Approach

The process can be:

1,000,000 patient records

↓

Divide into patient strata

↓

Perform proportional stratified random sampling

↓

Preprocess and clean the sample

↓

Train a suitably complex model

↓

Validate using cross-validation

↓

Check accuracy ≥ 95%

↓

Check confidence ≥ 95%

↓

Evaluate on an independent test set

10. Comparison of Sampling Methods

Method	**Advantage**	**Limitation**	**Suitability**
Simple Random Sampling	Simple and unbiased in large populations	May miss rare categories	Medium
Stratified Sampling	Represents all important categories	Requires identifying strata	**High**
Monte Carlo Sampling	Useful for simulation and uncertainty estimation	Not ideal for ensuring category representation	Low/Medium

Conclusion

Stratified Random Sampling is the best choice because the healthcare dataset contains different patient categories and the sample must represent all of them. It reduces sampling bias while

lowering memory and computational requirements. Sample complexity determines how much data is needed, VC dimension controls model complexity, and Occam's principle helps prevent unnecessary overfitting. Proper validation, calibration, and representative sampling can help achieve the required 95% accuracy and 95% confidence while working within the 8 GB RAM constraint.